Group #:
G 08

Design Project 1

ENSC 350 1267

Project Submitted in Partial Fulfillment of the Requirements for Ensc 350
Towards a Bachelor Degree in Engineering Science.

First Name: Duncan
Last Name: Miller
SID: 3 0 1 5 8
7 7 0 7

First Name: Laxmi
Last Name: Unnithan
SID: 3 0 1 3 9
3 1 1 5

First Name: Nazim Mete
Last Name: Balci
SID: 3 0 1 5 7
8 2 9 3

First Name: Serene
Last Name: Estacio
SID: 3 0 1 5 9
2 7 5 8

Intentionally left blank

© Lakshman One, 2026

Table of Contents

Project Scope and Specifications ____ 3

Design Architecture and Candidates ____ 4

- Baseline Ripple Carry Adder ____ 4
- Fast Ripple Adder ____ 4
- Conditional Sum Adder ____ 5
- Multi-Level Carry Lookahead Adder ____ 5
- Carry-Select Adder ____ 6
- Carry Skip Adder ____ 6

Functional Verification ____ 7

Experiments and Resource Analysis ____ 8

Normalization ____ 11

Conclusion ____ 12

Project Scope and Specifications

This design project focusses on creating and developing the baseline device as well as three other candidates for a 64-bit adder. The objective for this project is to evaluate these design candidate implementations for different Field Programmable Gate Array (FPGA) devices and analyze the trade-offs for each device and implementation. This is done through assessing two key factors – Cost and Timing.

Design Architecture and Candidates

Baseline Ripple Carry Adder

The implementation of our ripple carry adder (RCA) was designed through formal port mapping and generate statements to instantiate the one-bit full adder for a full 64-bit design requirement. Each 1-bit stage i processes the inputs A_i and B_i, alongside the carry-in bit C_{in_i}. Each bit position can be modeled with the following Boolean expressions:

$$S_i = A_i \oplus B_i \oplus C_{in_i}$$

$$C_{out_i} = (A_i \cdot B_i) + \left(C_{in_i} \cdot (A_i \oplus B_i)\right)$$

Within project framework, the baseline VHDL code is modeled structurally. The 1-bit full adder is instantiated N times (N = 64). When synthesizing target-independently on Altera/Intel FPGA boards, the synthesis tool maps the structural gate connections onto the device's dedicated hardware, making carry chains. Although the hardware routing *minimizes* the inter-stage interconnect delay, the N-stage linear ripple chain is one of the main tradeoffs for the maximum operating frequency.

Fast Ripple Adder

The FastRipple Adder uses a behavioral VHDL architecture that performs a 64-bit addition using the "+" operator rather than instantiating individual full adders. The input operands are extended by one bit using zero-padding to accommodate the final carry-out. The *sum_ext* signal calculates the addition of both operands and the incoming carry (Cin).

The lower N bits of *sum_ext* produce the sum(S), while the most significant bit provides the carry out (Cout). Signed overflow is detected by comparing the sign bits of the operands with the resulting sum.

Conditional Sum Adder

This design candidate focusses mainly on the principle of its recursive logarithmic tree carry network. In comparison to the baseline ripple adder, instead of waiting for the carry-bit to ripple across 64 bits sequentially, this design splits the N-bit word into two equal halves (M=N/2) to compute conditional results concurrently.

Within our design, we have two distinct cases controlled by VHDL generate statements:

1. Recursive case (N > 2): Instantiates a lower adder block to compute lower sum bits and the carry-select, alongside two concurrent upper blocks that pre-calculate the results for $C_{in} = '0'$ and $C_{in} = '1'$. This is the connected to a multiplexer that uses the carry select to select the correct upper-half results.
2. Stopping case (N = 2): This case terminates the recursion at 2-bit slices by using gate-level XOR, AND, and OR boolean expressions.

The top-level signals map out the internal vectors to output ports without violating the VHDL port-read rules. The signed two's complement overflow is also evaluated at the most significant bit (N-1) through comparing operand polarities with the final result sign, and can be modelled by the following boolean expression:

$$Ovfl = \left(A_{N-1} \cdot B_{N-1} \cdot \overline{S_{N-1}}\right) + \left(\overline{A_{N-1}} \cdot \overline{B_{N-1}} \cdot S_{N-1}\right)$$

When synthesizing for Altera/Intel FPGA architectures, Quartus flattens and synthesizes these recursive entity instantiations into dedicated FPGA logic. The main tradeoff for this implementation is the increase of Logic Elements (LE)/Adaptive Logic Module (ALM) resource consumption in exchange for a reduction in the propagation delay ().

Multi-Level Carry Lookahead Adder

The carry lookahead adder improves the baseline ripple carry addition by computing carry signals in parallel using local Generate ($G_i = A_i \cdot B_i$) and Propagate ($P_i = A_i \oplus B_i$) logic. Inputs A and B are split into 4-bit blocks, computing bitwise and terms alongside the group signals – P_{group} and G_{group} - which is done through the Lookahead Carry Generator (LACG) blocks.

For our 64-bit design (N=64), our group chose to use 16 of these 4-bit LACG blocks instead of 16-bit LACG blocks. This is because it reduces the gate complexity and avoids

high fan-in delays. On FPGA targets such as the Cyclone IV, it is beneficial to use the 4-bit blocks as it maps onto the devices 4-input look up tables (LUTs) cleanly, where a 16-bit LACG exceeds the LUT limit and results in slower multi-level decomposition.

The resulting group signals are then fed into 4 4-bit lookahead carry units (LCUs) along with the systems and directly evaluate the intermediate block carries () and the overall . This look-ahead carry relationship is modelled by the following boolean expression:

$$C_{i+1} = G_{group,i} + C_i \cdot P_{group,i}$$

The tradeoffs for this design candidate are that the multiple block usage increases the gate count as well as the routing complexity. On the FPGA architectures, although the look-ahead prediction quickens the logic, the multi-level LCU logic results in high fan-out routing delays relative to the dedicated, optimized FPGA hardware carry chains.

Carry-Select Adder

The carry-select design candidate mitigates the linear delay of the baseline adder by making four 16-bit blocks to account for the 64 bits. Each block independently computes the results in parallel under the two assumptions of Cin = '0' (Path 0) and Cin = '1' (Path 1).

We designed this implementation by using a parameterized generate loop and iterating based on the bit ranges and splitting it up into the lower index and higher index ranges. This loop has two concurrent behavioural processes to evaluate unsigned addition, one block for the assumption of Cin = '0' and the other for Cin = '1'. Each block has its own carry chain, which feeds into a multiplexer that instantly routes the correct sum and carry-out to its subsequent stages. The final top-level ports that output (S, Cout, and signed two's complement Overflow) are driven from internal signals to prevent VHDL port-read violations.

Our design for the carry-select adder reduces the worst-case timing delay compared to the baseline as it substitutes the rippling stages with parallel block paths and multiplexer selection trees that are quicker. However, a tradeoff with this design is that it requires duplicated hardware logic (the concurrent 0 and 1 adders for each 16-bit block) alongside the multiplexer routing, which results in higher LE utilization on the target FPGAs.

Carry Skip Adder

Our design of the carry skip adder reduces the worst-case latency by using a block-level bypass shortcut. The N-bit adder is instantiated into 4-bit ripple blocks, where within each block, if every individual bit position satisfies the propagate condition of $P_i = A_i \oplus B_i = 1$, the carry-in can bypass the entire block instantly and propagates directly to the next block without waiting for the gate-level ripple delays.

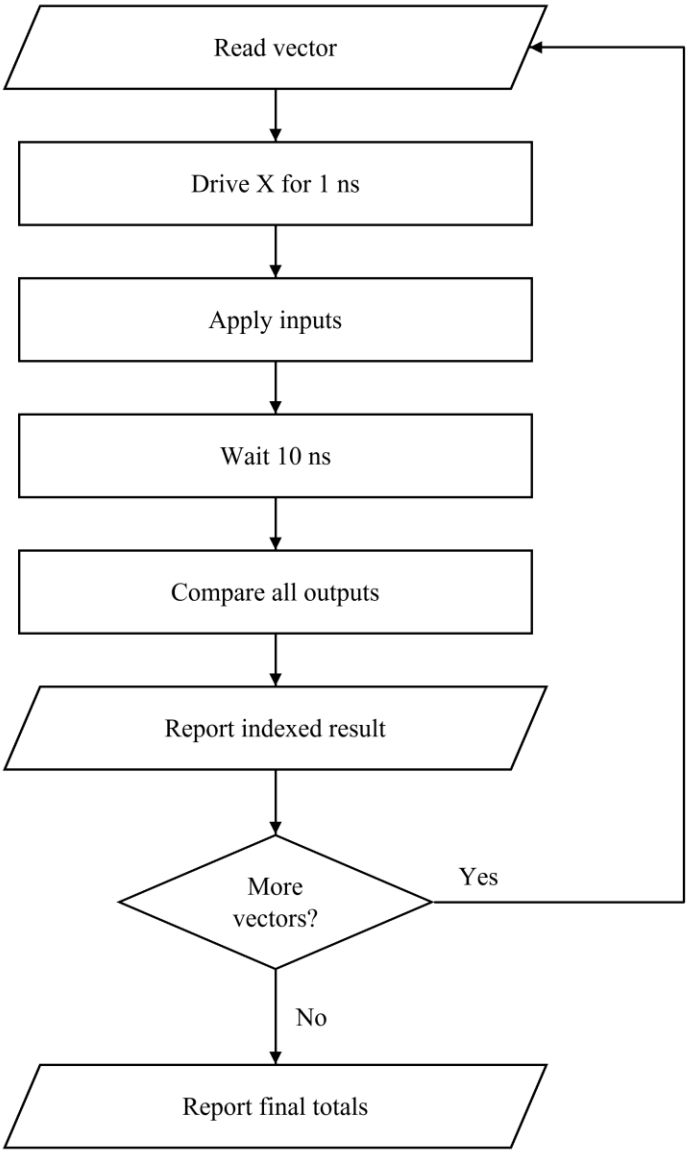
Within our design, our generate loops compute the number of blocks while an inner loop manages the 4-bit blocks. A conditional generate statement handle the arbitrary word width. We also use a chain vector that uses a bit-wise logical AND across all local propagate signals that drive a concurrent bypass multiplexer assignment. Internal signals safely manage the sum vectors, block carry paths, and signed twos complement overflow before outputting the top-level ports.

The main benefit for this design is that it accelerates the propagation delay and utilizes less hardware resources than the carry-select or carry-lookahead topologies. However, the limit to this design is that the blocks that do not satisfy the propagate condition, are bounded by internal ripple delays, and not achieving the logarithmic scaling.

Functional Verification

Functional verification was performed on the 64-bit structural ripple and conditional-sum architectures using ModelSim. The STIM process reads each measurement in the order A B Cin S Cout Ovfl. The operands and expected sum are represented by hexadecimal digits, while carry-in, carry-out and overflow are represented in binary.

Before applying each vector, the testbench drives all inputs to X for 1 ns. After applying the specified inputs, it allows 10 ns for the functional RTL model before checking S, Cout, and Ovfl.



Test Vector Selection

Expected results were generated from Python code using integer arithmetic. The lower 64 bits of the extended result define the expected sum, and remaining defines the carry out. There are a total of 1236 measurements that have been done. Both baseline and conditional sum pass all measurements correctly.

Experiments and Resource Analysis

The adder architectures are synthesized using Quartus Prime to evaluate their hardware resource requirements on Cyclone IV and Arria II FPGAs. The Baseline Ripple Adder serves as the reference for comparing the predicted and actual resource utilization of the alternative designs. Cyclone IV logic elements(LEs) and Arria II Adaptive Logic Modules(ALMs) are recorded and normalized to provide a consistent comparison.

Analysis of Cost

The hardware cost of each design candidate is evaluated based on the number of FPGA logic resources required to implement the architecture. Cyclone IV 4-input Look up tables(LEs) are used as the standard unit of cost.

ALM- LE conversion factor

Based on the Cyclone IV and Arria device handbooks, a conversion factor of 1.5 LEs per ALM was selected. While an Arria II ALM can implement two suitable LUT functions, its actual resource efficiency depends on logic packing, shared inputs and circuit structure. Therefore, a factor of 1.5 provides a conservative estimate rather than assuming ideal two to one mapping.

Predicted Cost

Resource cost predictions were based on the Intel Cyclone IV and Arria II device handbooks, where a Cyclone IV contains one 4-input LUT and supports a 2-bit full adder structure in arithmetic mode, while an Arria II ALM contains two dedicated full adders.

Baseline Ripple Adder

The Baseline consists of 64 interconnected one bit full adders. Assuming 2 addition bits per LE and on extra LE for overflow, the predicted cost is (64/2) +1 =33 LEs

Fast Ripple Adder

FastRipple uses the VHDL + operator, allowing Quartus to use the FPGA's built in arithmetic hardware. Using the same assumption, the cost is 33 LEs.

Conditional Sum Adder

The recursive architecture divides each adder into three smaller adders until reaching the 2 bit base case, resulting in 3^5 = 243 two bit adders. The five recursive levels require 543 sum/carry selection functions, giving 787 LEs(243+543+1) on Cyclone IV and 516 ALMs (243+272+1) or 774 LEs on Arria II.

Carry LookAhead Adder

The 64 bit Carry Lookahead adder is designed using 16 4-bit carry generator blocks that calculate generate and propagate signals, followed by higher-level carry logic. Assuming one Cyclone IV LE per separately mapped logic function, the estimated cost is 241 LEs (64 generate, 64 propagate 64 sum, 32 group functions, 16 carry functions, 1 overflow) or 121 ALMs on Arria II.

Carry Select Adder

The 64 bit Carry select adder consists of four 16-bit blocks, each calculating two possible results for carry- in values of 0 and 1. This gives 128 addition bit positions, requiring an estimated 64 arithmetic resources , plus 68 multiplexers, and one overflow resource, giving 133 LEs on Cyclone IV and approximately 99 ALMs on Arria II.

Carry Skip Adder

The 64 bit Carry Skip Adder consists of sixteen 4-bit ripple blocks, each containing carry propagation logic and a bypass path. Assuming two addition bits per arithmetic resource, the predicted cost includes 32 resources for addition,16 for block propagate logic, 16 for carry bypass selection, and one for overflow detection, giving 65 LEs for Cyclone IV(32+16+16+1) and 49 ALMs on Arria II.

Table of Cost

Design Candidates	**Predicted Cost**	**Actual Cost**
Baseline Ripple Adder	33 LEs	155 LEs
Fast Ripple Adder(Cyclone IV)	33 LEs	67 LEs
Fast Ripple Adder(Arria II)	50 LEs	50 LEs
Conditional Sum Adder(Cyclone IV)	787 LEs	262 LEs

Conditional Sum Adder(Arria II)	774 LEs	203 LEs
CLA (Cyclone IV)	241 LEs	13 LEs
CLA (Arria II)	182 LEs	14 LEs
Carry Select (Cyclone IV)	133 LEs	137 LEs
Carry Select (Arria II)	149LEs	167 LEs
Carry Skip(Cyclone IV)	65 LEs	224 LEs
Carry Skip(Arria II)	98 LEs	108 ALMs LEs

Normalization

To compare the hardware resource costs of each design candidate, we will analyze how each topology consume a varying amount of FPGA LEs, different routing interconnects, logic cells, and compare the numbers of each and how it contributes to the cost of each design.

To provide a relative comparison, each resource cost will be normalized against the Baseline Ripple-Carry Adder, which is the reference point. The normalized cost metric is modelled by the following equation:

$$C_{norm} = \frac{Candidate\ LE\ Count}{Baseline\ LE\ Count}$$

Architecture	**Total Logic Elements (LE)**	**Normalized Cost (C_{norm})**	**Area Overhead (%)**
Baseline Ripple-Adder	155	1.00 X	0
Fast Ripple Adder (Cyclone IV)	67	0.43 X	-56.8
Fast Ripple Adder (Arria II)	50	0.32 X	-67.7
Carry-Skip (Cyclone IV)	224	1.45 X	44.5
Carry-Skip (Arria II)	108	0.7 X	-30.3

Carry-Lookahead (Cyclone IV)	266	1.72 X	71.6
Carry-Lookahead (Arria II)	14	0.09 X	-91.0
Carry-Select (Cyclone IV)	137	0.88 X	-11.6
Carry-Select (Arria II)	167	1.08 X	7.7
Conditional-Sum (Cyclone IV)	262	1.69 X	69.0
Conditional-Sum (Arria II)	203	1.31 X	31.0

By using the baseline as the reference, normalization can show the price paid at the cost of speed enhancements. The advanced parallel carry structures on the Cyclone IV, such as the Carry-Lookahead and Conditional-Sum adders show an area penalty, as they require more LEs, demonstrating that there is an increase in hardware cost when eliminating the ripple carry slowdown. The cost is also dependent on the FPGA architecture, the Cyclone IV demands an increased hardware overhead to have a complex acceleration, the Arria II uses the advanced adaptive logic modules (ALM) to reduce the resource consumptions, as shown when comparing the LE in the carry-lookaheads for both FPGAs.

Conclusion

This project achieved designing, implementing and evaluating five distinct 64-bit adders, baseline ripple-carry, carry-select, carry-skip, multi-level carry-lookahead and conditional sum using VHDL for target FPGA boards. These design candidates were properly evaluated and analyzed by highlighting the trade-offs between the propagation delay (speed) and hardware resource utilization (area).

Our findings for the baseline and carry-skip design candidates have minimal logic and routing complexity which are efficient for designs that are constrained by area; however, are bounded by ripple delays. The carry-select, carry-lookahead and conditional sum candidates minimize the linear delay through using parallel carry prediction and concurrent computation, reducing the worst-case latency at the cost of higher LE count, multiplexer routing, and more fan-out complexity.

Finalizing the choice of adder topology rely on the system constraints, high-performance applications can benefit from the parallelism of the carry prediction, though resource-limited FPGAs, such as the Cyclone IV, benefit from the optimized block-skip or baseline topologies.